MÉTODO EQUALS

Comparación de objetos en Java

Desarrollo de Sistemas Orientado a Objetos · 2026

Prof. Esteban Santini · 2026

FIRMA Y OBJETIVO

public boolean equals(Object o)

El método equals compara el estado interno de dos objetos. Recibe un Object como parámetro porque puede compararse con cualquier tipo de objeto, incluso null.

Objetivo: Retorna true si ambos objetos son de la misma clase y absolutamente todos sus atributos tienen el mismo valor. Retorna false en cualquier otro caso.

Todas las clases heredan equals de Object. Sobrescribirlo (@Override) permite definir qué significa "igualdad" para tu propia clase.

ESTRUCTURA LÓGICA DEL MÉTODO

PRIMERAS COMPROBACIONES

- **1. Mismo objeto (this)**

Si o == this → retorna true. Un objeto siempre es igual a sí mismo.

- **2. Referencia nula**

Si o == null → retorna false. La nada no puede ser igual a algo.

Estas dos comprobaciones son atajos rápidos. Si pasan, procedemos a comparar en detalle.

IDENTIFICAR Y COMPARAR

- **3. Misma clase**

Si getClass() difiere → retorna false. No se comparan personas con sillas.

- **4. Castear y comparar atributos**

Castear Object al tipo propio y comparar cada atributo. Si alguno difiere → false. Si todos coinciden → true.

El casteo es seguro porque en el paso 3 ya verificamos que son de la misma clase.



Clase Person — Implementando equals

```java
public class Person {
    @Override
    public boolean equals(Object o) {
        if ( o == this ) return true;
        if ( o == null ) return false;
        if ( o.getClass() != this.getClass() ) return false;
        Person p = (Person) o;
        if ( !this.firstName.equals( p.firstName ) ) return false;
        if ( !this.lastName.equals( p.lastName ) ) return false;
        if ( this.age != p.age ) return false;
        else return true;
    }
}
```

EXPLICACIÓN: LÍNEAS 3 Y 4

Comparación con this y con null

La línea if (o == this) return true; compara referencias de memoria: si o y this apuntan al mismo espacio de memoria, son exactamente el mismo objeto. Retornar true de inmediato es correcto y eficiente.

Línea 4: if (o == null) return false; evita NullPointerException. Una referencia nula nunca puede ser igual a un objeto existente, así que se retorna false de forma segura.

Estas dos verificaciones son atajos de salida rápida. Si el objeto recibido pasa ambas, avanzamos a comparaciones más costosas.

EXPLICACIÓN LÍNEA A LÍNEA

@Override

Indica que se está reescribiendo (sobreescribiendo) el método equals que toda clase hereda de Object. Sin esta anotación el compilador no avisa si cometemos un error en la firma.

Firma del método

Recibe un Object porque así puede compararse con cualquier instancia, sin importar la clase. Retorna boolean: true si los objetos son iguales, false si no.

Casteo (línea 6)

Como ya sabemos que o es de la misma clase (paso 3), es seguro castearlo: Person p = (Person) o; Esto nos da acceso a los atributos de la clase.

Comparar atributos

Comparar cada atributo: primitivos con != (por ejemplo age), objetos con .equals() negado (por ejemplo firstName). Si alguno difiere, retornar false.

COMPARAR ATRIBUTOS Y SIMPLIFICAR

Distintas formas de hacer la última comparación

Primitivos (int, double...)

Usar el operador != directamente. Ejemplo: if (this.age != p.age) return false;

Objetos (String, ArrayList...)

Usar .equals() negado. Ejemplo: if (!this.firstName.equals(p.firstName)) return false;

Simplificación posible

Las líneas if/else del final se pueden simplificar retornando directamente el resultado: return this.age == p.age;



EQUALS: COMPARAR ESTADO INTERNO

"Dos objetos son iguales cuando su estado interno es idéntico, no cuando ocupan el mismo lugar en memoria."
equals() bien implementado garantiza comparaciones correctas. Sin él, Java compara referencias (==), no contenido.